Authored by: Arpit Raj, LNMIIT Jaipur
Date: October 27, 2026
© 2026 Arpit Raj. All rights reserved.

Compile-Time Polymorphism & Operator Overloading in C++

POLYMORPHISM

- -> a phenomenon that allows an object to have several different forms and behaviours based on given context.
- -> basically ability of a function to work in diff ways based on object it's acting upon.

we have already covered runtime polymorphism (dynamic).

Static Polymorphism / Compile time Polymorphism

-> can be achieved via function overloading or operator overloading.

Function Overloading Rules

-> defining multiple functions with same name but different parameter lists within the same scope.

Eg

```
int add(int a, int b) { return a+b; }
double add(double a, double b) { return a+b; }
int add(int a, int b, int c) { return a+b+c; }
```

-> valid overloads

- different number of params, different parameter type, different parameter order are all allowed.
- different return type only and different parameter name only -> is not a valid overload.

it's compile time because: it deals with syntax and verifiability of program which is checked during compile time.

Operator Overloading Basics

Eg

```
class Complex {
public:
    int real, imag;
    Complex(int r, int i) { real = r; imag = i; }

    Complex operator+(Complex c) {
        Complex temp(real + c.real, imag + c.imag);
        return temp;
    }
};
```

now in main if we write

```
Complex C1(2, 3);
Complex C2(4, 5);
Complex C3 = C1 + C2;
```

then `C3 = 6 + 8i`

`C3 = C1 + C2` may be interpreted as:-
`C3 = C1.operator+(C2)`
now that function `operator+` is the key player here.

- `C1` is object that calls the member function
- `C2` is passed as argument
- the function adds
- returns a new complex object

RULES FOR OPERATOR OVERLOADING

- cant invent a new operator
- cant change precedence order
- cant change number of operands an operator takes

© Arpit Raj, LNMIIT Jaipur

Need of operator overloading

```
int a=10;
int b=20;
int c=a+b; // C++ knows how to add integer
```

but if we create our own class

```
class Num {
public: int val;
};

Num a, b;
Num c = a+b; // then No suitable operator+ is defined
```

C++ dont know how objects are added.

Solution

```
class Number {
    int value;
public:
    Number operator+(Number other) {
        return Number(value + other.value);
    }
};
```

this function needs to be written within Number class.

Member vs Non-Member Overloading

(A) Member function

```
Number operator+(Number other) {
    return Number(value +
other.value);
}
```

`Number c = a + b;` is equivalent to :
`Number c = a.operator+(b);`

(B) Non Member function

```
Number operator+(Number a, Number
b) {
    return Number(a.value +
b.value);
}
```

`Number c = a + b;` is equivalent to :
`Number c = operator+(a, b);`

- member overload has access to private members but non member overload is not having access unless given via friend

#

```
Number a(10);
Number b = a + 5;
Number c = 5 + a;
```

- Non member overload supports both operand orders
- in a member overload (`5+a`) cant invoke `operator+` because the left operand `5` is not an object which can call the function.
- Hence both have their own pro and con.

considering member overload

- `a + b` needs 2 operands, where 1 is explicit parameter
- `++a` needs 1 operand, 0 explicit parameter
- `a++` needs 1 operand, 1 dummy int parameter

Prefix vs Postfix Increment

```
Counter& operator++() {
    value++;
    return *this;
} // -> prefix ++a

Counter operator++(int) {
    Counter old(value);
    value++;
    return old;
} // -> postfix a++
```

- the extra `int` is a dummy parameter used to distinguish the postfix overload from prefix overload.
- `++a` returns updated object by reference.
- `a++` returns the old value, commonly by value.

#

```
bool operator==(const Number& other) const {
    return value == other.value;
}
```

in equality operators it is good practice to use `const`. it avoids modifying the argument.
trailing `const` ensure calling object is not modified.

Overloading Stream Operators << & >>

assume

```
Number n(69);
cout << n << "\n" // this is not valid
```

to make it valid we need to write :- a custom function (operator overloading) inside Number class

```
friend ostream& operator<<(ostream& cout, const Number& n) {
    cout << n.value;
    return cout;
}
```

> 💡 **Why ostream& operator<< must be a Non-Member (friend)**
>
> - we need to use a non member overload because left operand is `cout`.
> - `cout` type is `ostream` and not `Number` so it cant call.
> - `friend` allows access to value in case private.
> - `return cout` allows using multiple Eg `cout << a << b << aadya << "\n"`.

Assignment & Other Operators

```
Number& operator=(const Number &other) {
    value = other.value;
    return *this;
}
```

we return `Number&` because it allows chained assignments like `a = b = c`.
-> `b = c` returns `b` by reference and then `a = b` executes.

one rule

- -> at least one operand must have a class, and we cant change number of operands.

`#int& operator[](int index);`
support syntax such as `arr[2] = 10` for an object.

`#int operator()(int x);`
supports `obj(5);`